

小车巡线

小车巡线

- 1.快速上手
- 2.硬件接线
- 3.使用方法
- 4.现象结果
- 5.代码解释

MotorDriver 类

GrayscaleSensor 类

LineFollowing 类

1.快速上手

本教程讲解如何使用PICO2主板在获取八路灰度巡线模块的数据之后开始在赛道上巡线，以及如何修改代码中的部分参数，使小车在用户自己的赛道场景中运行的效果更好。

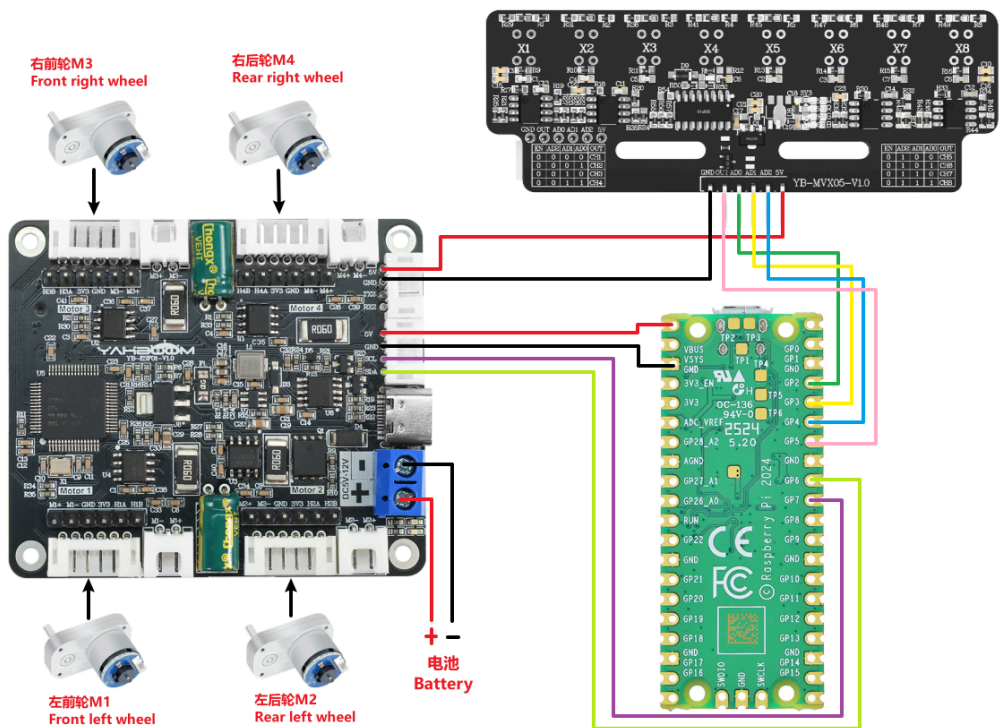
教程中使用的硬件均是亚博售卖的PICO2、八路灰度巡线模块、轮式小车底盘V2、L型520电机、四路电机驱动板、12V电池组，查看下方的接线图进行接线后，再给PICO2烧录程序，就可以上电开始巡线了，如果效果不佳，可以跳转到代码讲解章节，查看LineFollowing类的说明，对巡线参数进行修改，优化巡线效果。

除此之外，还需要修改合适的电机PID参数，本案例中使用的是L型520电机，使用的电机PID值为：P:1.9,I:0.2,D:0.8。使用PC串口助手与四路电机驱动板通信，发送指令修改PID值，如果使用的是其他电机，需要自行测试出一个合适的电机PID参数。

此调优步骤需要用户具备一定的调试小车经验。非亚博的模块仅供参考。

2.硬件接线

由于PICO2主板上的5V引脚不够使用，所以八路灰度巡线模块的5V和GND可以接在四路电机驱动板上。



八路灰度巡线模块	PICO2
AD0	GP2
AD1	GP3
AD2	GP4
OUT	GP5

八路灰度巡线模块	四路电机驱动板
5V	5V
GND	GND

四路电机驱动板	PICO2
5V	VBUS
GND	GND
SCL	GP7
SDA	GP6

亚博售卖的八路灰度巡线模块标配的线材为XH2.54转6pin杜邦线，XH2.54排线接口一端要接在八路灰度模块上，另一端的杜邦线则与上图一样正常连接即可：



亚博售卖的四路电机驱动板与L型520电机的连接方法可以到四路电机驱动板资料中找到《常用电机介绍以及用法》的教程，查看详细的具体接线方式，由于篇幅较长，这里不再展开。

3.使用方法

找到代码 Grayscale_Trace ，使用Thonny软件打开，接入树莓派pico2后，点击上方工具栏最右侧的停止键：

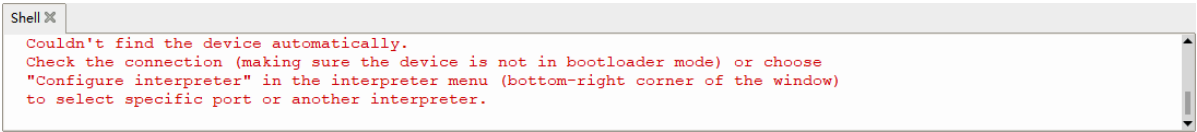


然后可以看见最下方的消息栏中弹出pico2中当前固件的信息，这样代表软件已经识别到了串口：

```
Shell ❸
>>>

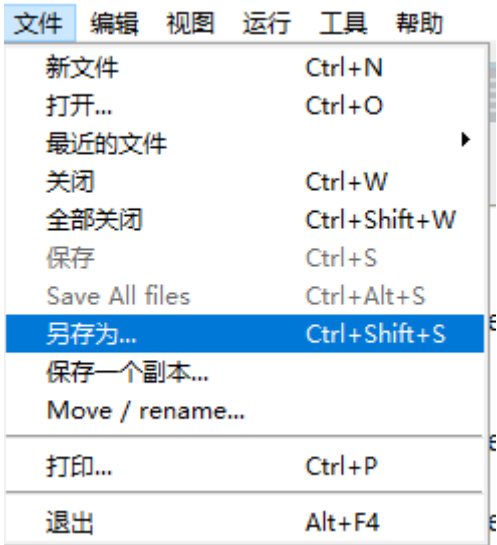
MicroPython v1.24.0-preview.201.g269a0e0e1 on 2024-08-09; Raspberry Pi Pico2 with RP2350
Type "help()" for more information.
>>> |
```

如果这里的消息栏显示无法识别串口，或者不会基础操作使用的用户，则需要查阅PICO2主板的资料，找到开发环境搭建（python）的教程，学习相关的基础使用以及烧录固件。给主板刷入固件就能识别串口了：



由于小车巡线无法让PICO2一直接着数据线运行，所以这里需要将程序写入到PICO主板中，离线运行。

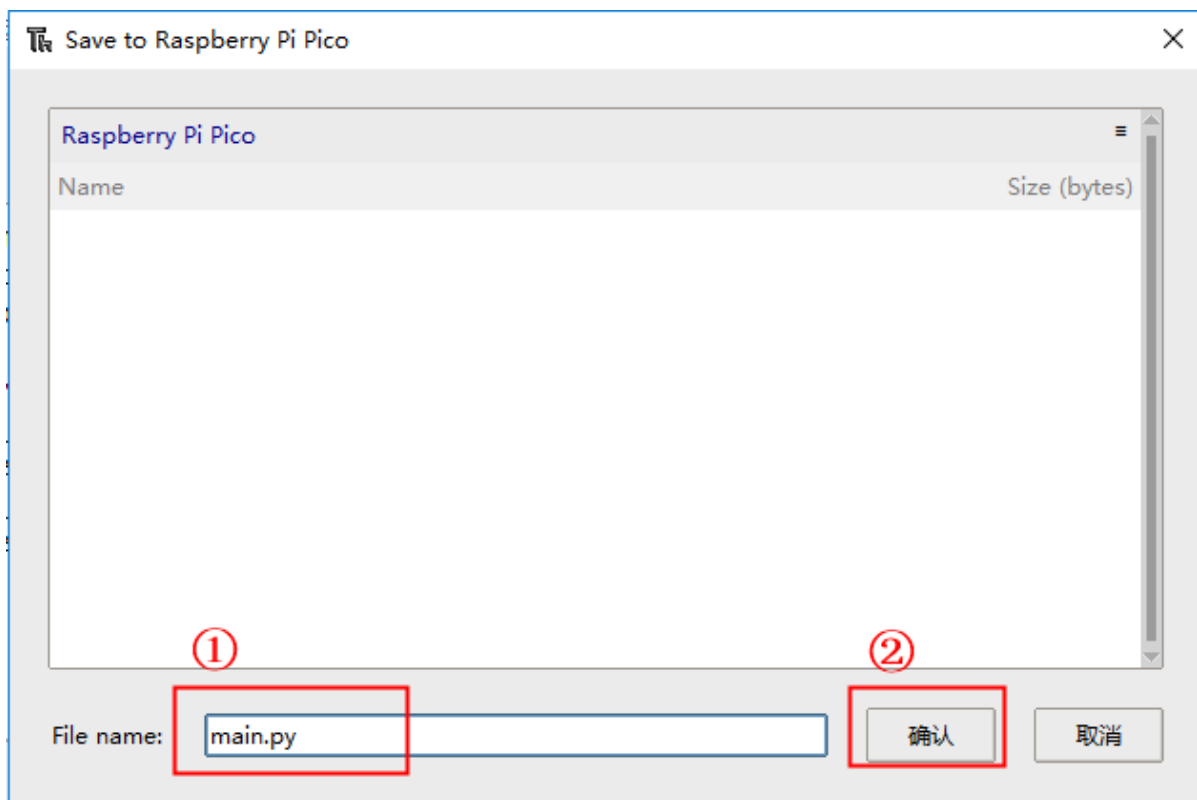
1. 选择 文件--另存为。



2. 选择 Raspberry Pi Pico。



3. File name 里输入 main.py，注意这里必须是main.py的文件名才能启动，然后点击 确认。



4. 等待程序烧录进度条完成后，就可以断开连接，上电让小车跑起来了。
5. 如果需要删除不再自动运行这个程序，则再次点击另存为，进入PICO，然后将已经保存在里面的main.py右键进行删除即可。

4.现象结果

上电之后，PICO2的板载LED灯会闪烁三次，代表成功上电，等待几秒后，板载LED常亮，说明硬件接口初始化完毕，开始巡线，此时如果八路灰度巡线模块上的八个LED灯都是全亮或者全灭，小车则不会动，防止因为被放在了错误的地图上乱跑，当巡线模块上的灯不一致时，小车开始巡线。

在要巡的线上的巡线模块LED灯，需要和线外的赛道相反，也就是说假如线上的LED灯是灭的，线外面的赛道上对应的LED灯需要亮起，才能正常巡线。

5.代码解释

```
#####
# 重要配置参数 Important Configuration Parameters
#####
# 根据自己使用的电机类型，选择对应的电机参数配置
# Select the corresponding motor parameter configuration according to the type of
motor you use
MOTOR_TYPE = 5 #1:520电机 2:310电机 3:测速码盘TT电机 4:TT直流减速电机 5:L型520电机
               #1:520 motor 2:310 motor 3:speed code disc TT motor 4:TT DC
reduction motor 5:L type 520 motor
#####
# 将灰度巡线模块放置于赛道，观察正对着线的灯，如果灯亮，则数值为1，LINE_RAW_VALUE=1；如果灯
灭，则数值为0，LINE_RAW_VALUE=0
# Place the grayscale line-following module on the track and observe the light
facing the line. If the light is on, the value is 1, LINE_RAW_VALUE=1; if the
light is off, the value is 0, LINE_RAW_VALUE=0
LINE_RAW_VALUE = 1
#####
```

代码开头的地方有 `MOTOR_TYPE` 和 `LINE_RAW_VALUE` 两个宏定义设置。

- `MOTOR_TYPE`：根据自己所购买的电机修改`MOTOR_TYPE`，代码中适配的电机都是亚博目前在售的电机。
- `LINE_RAW_VALUE`：需要将灰度巡线模块放置于赛道，观察正对着线的灯，如果灯亮，则填`LINE_RAW_VALUE=1`；如果灯灭，则数值为0，`LINE_RAW_VALUE=0`。

```
class MotorDriver:
    def __init__(self):
        # ...

        # 配置电机类型 Configure motor type
        def set_motor_type(self, data):
            self.i2c_write(self.MOTOR_MODEL_ADDR, self.MOTOR_TYPE_REG, [data])

        # 配置死区 Configuring Dead Zone
        def set_motor_deadzone(self, data):
            buf = [(data >> 8) & 0xFF, data & 0xFF]
            self.i2c_write(self.MOTOR_MODEL_ADDR, self.MOTOR_DEADZONE_REG, buf)

        # 配置磁环线 Configuring magnetic loop
        def set_pluse_line(self, data):
            buf = [(data >> 8) & 0xFF, data & 0xFF]
            self.i2c_write(self.MOTOR_MODEL_ADDR, self.MOTOR_PLUSELINE_REG, buf)

        # 配置减速比 Configure the reduction ratio
        def set_pluse_phase(self, data):
            buf = [(data >> 8) & 0xFF, data & 0xFF]
            self.i2c_write(self.MOTOR_MODEL_ADDR, self.MOTOR_PLUSEPHASE_REG, buf)

        # 配置轮子直径 Configuration Diameter
        def set_wheel_dis(self, data):
            bytes_data = self.float_to_bytes(data)
            self.i2c_write(self.MOTOR_MODEL_ADDR, self.WHEEL_DIA_REG,
list(bytes_data))

        # 控制速度 Controlling Speed
        def control_speed(self, m1, m2, m3, m4):
            speeds = [
                (m1 >> 8) & 0xFF, m1 & 0xFF,
                (m2 >> 8) & 0xFF, m2 & 0xFF,
                (m3 >> 8) & 0xFF, m3 & 0xFF,
                (m4 >> 8) & 0xFF, m4 & 0xFF
            ]
            self.i2c_write(self.MOTOR_MODEL_ADDR, self.SPEED_CONTROL_REG, speeds)

        # 控制PWM(适用于无编码器的电机) Control PWM (for motors without encoder)
        def control_pwm(self, m1, m2, m3, m4):
            pwms = [
                (m1 >> 8) & 0xFF, m1 & 0xFF,
                (m2 >> 8) & 0xFF, m2 & 0xFF,
                (m3 >> 8) & 0xFF, m3 & 0xFF,
                (m4 >> 8) & 0xFF, m4 & 0xFF
            ]
            self.i2c_write(self.MOTOR_MODEL_ADDR, self.PWM_CONTROL_REG, pwms)

        def set_motor_parameter(self):
            # ...
            pass
```

MotorDriver 类

- `set_motor_type` : 配置电机驱动板的电机类型。
- `set_motor_deadzone` : 配置电机驱动板上电机的死区参数。
- `set_pluse_line` : 配置电机驱动板上光电编码器的磁环线数。
- `set_pluse_phase` : 配置电机驱动板上电机的减速比。
- `set_wheel_dis` : 配置电机驱动板上小车轮子的直径。
- `control_speed` : 通过I2C向电机驱动板发送指令，以编码器闭环控制四个电机的速度。
- `control_pwm` : 通过I2C向电机驱动板发送指令，以PWM开环控制四个电机的转速。
- `set_motor_parameter` : 根据预设的电机类型（MOTOR_TYPE），配置电机的相关参数（类型、减速比、磁环线、轮径、死区）。

```
# 灰度传感器类 Grayscale Sensor Class
class GrayscaleSensor:
    def __init__(self, ad0_pin=2, ad1_pin=3, ad2_pin=4, out_pin=5):
        # ...

    def _select_channel(self, channel):
        """
        选择传感器通道
        Select the sensor channel
        """
        self.ad0.value((channel >> 0) & 0x01)
        self.ad1.value((channel >> 1) & 0x01)
        self.ad2.value((channel >> 2) & 0x01)

    def _read_out_value(self):
        """
        读取OUT引脚的值
        Read the value from the OUT pin
        """
        return self.sensor_out.value()

    def read_all(self):
        """
        读取所有通道
        Read all channels
        """
        values = []
        for i in range(self.channels):
            self._select_channel(i)
            time.sleep_us(50)
            values.append(self._read_out_value())
        return values
```

GrayscaleSensor 类

- `_select_channel` : 根据通道号, 通过GPIO输出高低电平选择多路复用器的传感器通道。
- `_read_out_value` : 读取传感器数据输出引脚的当前数字值。
- `read_all` : 依次选择所有传感器通道, 读取并返回每个通道的输出值列表。

```
# 巡线类 Line Following Class
class LineFollowing:
    def __init__(self, motor, sensor):
        """
        初始化巡线控制器
        Initialize the line following controller
        """
        ...

    # PID参数 PID Parameters
    self.kp = 160.0    # 比例系数 Proportional coefficient - 提高响应速度
    self.ki = 0.5      # 积分系数 Integral coefficient - 增强稳定性
    self.kd = 20.0     # 微分系数 Derivative coefficient - 提高预判能力

    ...

    # 控制参数 Control Parameters
    self.base_speed = 330    # 基础速度 Base speed
    self.max_speed = 400     # 最大速度 Max speed

    # 传感器权重 Sensor weights (8个传感器的位置权重)
    self.sensor_weights = [-5.0, -4.0, -2.0, -1.0, 1.0, 2.0, 4.0, 5.0]

    ...

    def check_sensors_safe(self, sensor_values):
        # ...

    def calculate_error(self, sensor_values):
        # ... (code for calculate_error)
    def pid_control(self, error):
        # ... (code for pid_control)
    def differential_speed_control(self, pid_output):
        # ... (code for differential_speed_control)
    def follow_line(self):
        # ... (code for follow_line)
```

LineFollowing 类

优化巡线效果重点修改项:

PID 参数

kp: 调大 `kp` 可使响应更快, 但易震荡; 调小 `kp` 可减少震荡, 但响应会变慢且可能偏离路线。

ki: 调大 `ki` 可更快消除稳态误差, 但易引起超调和震荡; 调小 `ki` 可降低超调风险, 但消除误差变慢甚至无法完全消除。

kd: 调大 kd 可提升系统稳定性, 减少摆动和超调, 但过大会对噪声敏感并引发抖动; 调小 kd 可降低对噪声的敏感, 但抑制摆动能力减弱, 响应可能滞后或超调。

速度参数

base_speed: 决定小车在平稳巡线时的前进速度, 影响整体效率和弯道响应。

max_speed: 限制小车左右轮的最高转速, 防止小车加速偏差过大、过快。

灰度巡线模块权重参数

sensor_weights: 修改灰度巡线模块上的权重, 第一个值指代模块最边缘的X1灯, 第二个值为X2灯。第一个数值调大, 则在X1灯亮时反应更剧烈, 调小则平缓。

- `check_sensors_safe`: 检查所有灰度传感器是否都显示相同值, 用于在启动时判断小车是否处于安全状态。
- `calculate_error`: 根据灰度传感器读取的值和预设权重, 计算小车偏离线中心的误差值。
- `pid_control`: 执行PID控制算法, 根据输入的误差值计算控制输出, 包含死区处理、积分清零和动态积分限幅。
- `differential_speed_control`: 根据PID控制输出, 计算左右轮的差速, 从而调整小车转向, 并对速度进行限制。
- `follow_line`: 巡线主函数, 负责读取传感器、执行安全检查、计算误差、进行PID控制和差速控制来驱动电机。